Precision Controller
Interface Specification

LonghornSilicon LLM Inference Accelerator

Block 1 of 4 — Attention Compute Unit (ACU)

Version: `pc-isa-0.1` (first public draft)
Date: 2026-05-13
Status: Pre-tape-out stub for compiler integration

Author: Chaithu Talasila, The University of Texas at Austin

Reference: `LonghornSilicon/adaptive-precision-attention`

Scope. This document describes the externally-visible interface to the `precision_controller` block as it will appear on the chip, on an FPGA prototype (Xilinx ZCU102/104), or in the bit-accurate software model. A compiler backend targeting the precision controller writes against this interface; the hardware implementation must conform to it. This specification is stable for the precision controller block (ACU) only and will be unified with the rest of the LonghornSilicon ISA when the KV Cache Engine, Token Importance Unit, and Memory Hierarchy Controller blocks land.

Contents

1 Block Overview **3**

2 Address Space and Memory Map **3**

3 Streaming Data Interfaces **4**
3.1 `s_axis_scores` — Score Input (Slave) 4
3.2 `m_axis_decisions` — Decision Output (Master) 4

4 Logical Operations **4**
4.1 Worked Lowering Example 5
4.2 Compiler Binding Patterns 6

5 C API Stub **6**

6 Synthesis-Time Configuration **7**

7 Bit-Accurate Reference **8**

8 Integration Phases **8**

9 Open Questions **9**

10 Change Log **9**

1. Block Overview

The precision controller is a streaming block that consumes one signed attention score per cycle, asserts an end-of-tile signal on the last score, and produces a single-bit precision decision (INT8 or FP16) one cycle later. The decision is computed by the integer rearrangement of an entropy-equivalent ratio gate, requiring no division and no transcendentals:

$$\text{decision} \;=\; \big(\max(|s|) \ll \log_2 N\big) \;>\; \big((\Sigma(|s|) \ll 3) + (\Sigma(|s|) \ll 1)\big)$$

or equivalently,

$$\text{decision} \;=\; \max(|s|) \cdot N \;>\; \Sigma(|s|) \cdot 10$$

where decision = 1 means this tile needs FP16 and decision = 0 means it can use INT8. The constant $N = \text{BLOCK\_M} \cdot \text{BLOCK\_N}$ is the synthesis-time tile size; the threshold $T = 10$ is hard-coded in the synthesized netlist via the equivalent shift-and-add $(\Sigma \ll 3) + (\Sigma \ll 1)$. Changing either requires re-synthesis (see §6).

Latency: one cycle after `s_last` asserts.
Throughput: one score per cycle, one decision per N scores.
Footprint: 30 flip-flops, $\sim$3.4 k µm^2 stdcell at SkyWater Sky130A; projects to $\sim$150 µm^2 at TSMC 16FFC.
Bit-exact reference: `sw/reference_model/precision_controller_ref.py` (143/143 RTL replay tiles match).

2. Address Space and Memory Map

The block exposes a 256-byte AXI-Lite slave register window. All registers are 32-bit, word-aligned. The base address is set at chip integration time (e.g., `0x4000_0000` on the ZCU102 PYNQ overlay).

Table 1: AXI-Lite register window (offsets relative to base address).

Offset	Name	Access	Reset	Purpose
0x00	CTRL	RW	0x0	bit[0]: `soft_reset` (write-1 to pulse); bit[1]: `enable`
0x04	STATUS	R	0x1	bit[0]: `idle`; bit[1]: `decision_valid`; bit[2]: `tile_in_progress`; bit[3]: `fifo_full`
0x08	INFO_BLOCK_M	R	(syn)	Synthesis-time `BLOCK_M`
0x0C	INFO_BLOCK_N	R	(syn)	Synthesis-time `BLOCK_N`
0x10	INFO_N	R	(syn)	`BLOCK_M` · `BLOCK_N`
0x14	INFO_SCORE_WIDTH	R	(syn)	Bits per signed score on `s_axis_scores`
0x18	INFO_THRESHOLD	R	(syn)	Always 10 in `pc-isa-0.1`
0x1C	INFO_VERSION	R	0x100	bits[15:8] major, bits[7:0] minor
0x20	TILE_COUNT	R	0x0	Complete tiles processed since last reset
0x24	LAST_DECISION	R	0x0	bit[0]: most recent decision (1 = FP16)
0x28	DECISION_FIFO	R	—	Pop one entry; bit[0] = decision; bit[31] = `valid`
0x2C	DECISION_FIFO_LVL	R	0x0	Number of decisions waiting in the FIFO
0x30	IRQ_MASK	RW	0x0	bit[0]: enable interrupt on every decision
0x34	IRQ_STATUS	R/W1C	0x0	bit[0]: decision-available IRQ pending; write 1 to clear

Conventions. RW = read/write; R = read-only; W1C = write-1-to-clear; (syn) = value fixed at synthesis time. Writes to read-only registers are silently dropped. Reading reserved offsets returns zero.

3. Streaming Data Interfaces

Two AXI-Stream interfaces carry the actual scalar data. The AXI-Lite window in §2 is for control only.

3.1 s_axis_scores — Score Input (Slave)

Signal	Width	Direction	Purpose
tdata	SCORE_WIDTH	in	One signed two's-complement attention score
tlast	1	in	Assert on the final score of each tile
tvalid	1	in	Handshake: data is valid
tready	1	out	Handshake: block is ready to accept

Protocol:

- A complete tile is exactly N beats. The block does not validate the beat count; feeding fewer than N beats and asserting tlast mid-tile produces an undefined decision.
- tlast is asserted on the Nth beat. The decision is computed combinationally on that cycle's tdata (so the running max/sum sees the last score), and the decision is latched on the following rising edge.
- Backpressure: when the block is not ready (e.g., the decision FIFO is full), tready deasserts. Upstream must hold tdata and tlast stable until the handshake completes.

3.2 m_axis_decisions — Decision Output (Master)

Signal	Width	Direction	Purpose
tdata	1	out	bit[0]: decision (1 = FP16, 0 = INT8)
tvalid	1	out	Handshake: decision is valid
tready	1	in	Handshake: downstream is ready to accept
tlast	1	out	Always 1 (each decision is a one-beat packet)

Protocol. One beat per processed tile. If the downstream consumer is not ready, the decision queues in the internal FIFO (default depth 16). When the FIFO fills, the block stops accepting scores via tready on s_axis_scores.

4. Logical Operations

Below are the abstract operations a compiler emits. Each maps to a short sequence of register writes and stream beats; the C API in §5 gives the concrete library calls.

The block has no other externally-observable side effects. There is no clock-gating register, no power state machine, and no DFT scan-chain control at this stub level (those land on the chip-level ISA when the full chip is defined).

Operation	Inputs	Outputs	Description
PC_QUERY	—	INFO struct	Read all INFO_* registers in one transaction
PC_RESET	—	—	Soft reset accumulators + clear decision FIFO
PC_ENABLE	—	—	Set bit[1] of CTRL
PC_PUSH_TILE	N scores	—	Stream a tile via s_axis_scores with tlast on the final beat
PC_READ	—	decision bit	Pop one decision from the FIFO; blocks if empty
PC_READ_BATCH	count	count bits	Pop up to count decisions; non-blocking
PC_STATUS	—	STATUS bits	Read the STATUS register

4.1 Worked Lowering Example

This section walks through how a representative compiler IR operation lowers to a sequence of ISA operations. The example uses a generic high-level flash_attention op that any silicon-agnostic compiler is likely to have; the binding to a specific compiler IR (MLIR, TVM Relax, ONNX, etc.) is discussed in §4.2.

Suppose the compiler's IR contains:

```
%out = flash_attention(%q, %k, %v)
    : tensor<S x D x f16>, tensor<S x D x f16>, tensor<S x D x f16>
    -> tensor<S x D x f16>
```

The lowering proceeds in three stages. **Stage 1: setup.** The compiler queries the chip's tile dimensions once per compilation unit:

```
info = PC_QUERY()                  // -> block_m=64, block_n=64, n=4096
PC_RESET()
PC_ENABLE()
```

Stage 2: per-tile loop. For each (BLOCK_M, BLOCK_N) sub-block of the $Q \cdot K^\top$ matrix, the compiler emits a precision-routing decision before dispatching the actual matmul:

```
for (tile_i, tile_j) in tiles_of(Q, K):
    scores = Q[tile_i] @ K[tile_j].T          // FP16, computed in shared
        SRAM
    PC_PUSH_TILE(scores.flatten())            // stream N scores in
    decision = PC_READ()                      // 1 cycle after tlast

    if decision == FP16:
        out_tile = fp16_matmul(scores, V[tile_j])
    else:
        scores_int8 = quantize_int8(scores)
        out_tile = int8_matmul(scores_int8, V[tile_j])
    accumulate(out, out_tile)
```

Stage 3: batching (optional). For higher throughput the compiler can pre-queue many tiles into PC_PUSH_TILE and pop decisions in batches once enough have completed:

```
for tile in tiles[:k]:
    PC_PUSH_TILE(tile_scores(tile))      // tiles fan into the FIFO
decisions = PC_READ_BATCH(k)              // pop k results at once
dispatch_int8_tiles ([t for t,d in zip(tiles,decisions) if not d], V)
dispatch_fp16_tiles ([t for t,d in zip(tiles,decisions) if     d], V)
```

The same three stages work whether the compiler's target is the Python reference model (phase 0), the FPGA prototype (phase 1), or the silicon chip (phase 3). Only the runtime implementation of `PC_*` changes under the hood — Python function call, AXI register write, or PCIe MMIO, respectively. The lowered IR is identical.

4.2 Compiler Binding Patterns

The five logical operations in Table 1 cover every interaction a compiler needs with the precision controller. How those operations surface inside a given compiler depends on the compiler's IR. Sketches of the four most common patterns:

MLIR dialect. Define a `lhsi.pc.*` dialect with five ops corresponding to the table in §4. The lowering pass matches a `flash_attention` (or vendor-equivalent) op and rewrites it into the `lhsi.pc` sequence above plus the appropriate `linalg.matmul` variants. Conversion to LLVM IR happens in a subsequent pass via a runtime call interface.

TVM Relax / TIR. Register a backend target named `"lhsi-pc"`, add a pattern-matcher in the operator-fusion pass that recognizes attention sub-graphs, and emit calls to TVM-runtime extern functions named for each `PC_*` operation. The TVM runtime then dispatches to either the Python reference (test mode) or the C driver (deployed mode).

ONNX — graph rewriter. Add an ONNX-Runtime `CustomOpDomain` with one op per `PC_*`, then write a graph-transform pass that recognizes the attention pattern and inserts the custom ops in the right places. The runtime implementation of each custom op binds to either the Python reference or the C driver.

Custom IR. If the compiler has its own IR, the integration point is wherever its codegen emits the lowest-level hardware-target operations. Replace the call-site for the existing attention codegen with a sequence of calls to `PC_*` plus the precision-routed matmul kernels.

In every case, the contract on our side is the same: a compiler binding that emits the ISA operations defined in this document is correct, testable today against the Python reference, and forward-compatible through every integration phase.

5. C API Stub

What a runtime built on top of this ISA looks like. Concrete header sketch:

```
/* lhsi_precision_controller.h --- LonghornSilicon ACU driver API. */

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

typedef struct {
    uint32_t block_m;
```

```
    uint32_t block_n;
    uint32_t n;            /* = block_m * block_n */
    uint32_t score_width;  /* bits per score */
    uint32_t threshold;
    uint16_t isa_major;
    uint16_t isa_minor;
} lhsi_pc_info_t;

typedef struct lhsi_pc_handle lhsi_pc_handle_t;

/* Open / close */
int  lhsi_pc_open (const char *device, lhsi_pc_handle_t **out);
void lhsi_pc_close(lhsi_pc_handle_t *h);

/* Configuration queries (read INFO_* registers) */
int  lhsi_pc_query(lhsi_pc_handle_t *h, lhsi_pc_info_t *out);

/* Control */
int  lhsi_pc_reset (lhsi_pc_handle_t *h);
int  lhsi_pc_enable(lhsi_pc_handle_t *h, bool enable);

/* Streaming --- push a complete tile (N scores, score_width bits each).
   Blocks until the entire tile has been accepted by the block's tready
   handshake. */
int  lhsi_pc_push_tile(lhsi_pc_handle_t *h,
                       const void *scores, size_t stride);

/* Pop a single decision (blocks if FIFO is empty). */
int  lhsi_pc_read_decision (lhsi_pc_handle_t *h, bool *out_fp16);

/* Pop up to `count` decisions; returns number actually popped. */
int  lhsi_pc_read_decisions(lhsi_pc_handle_t *h,
                            bool *out_fp16, size_t count);

/* Diagnostics. */
typedef struct {
    bool idle;
    bool decision_valid;
    bool tile_in_progress;
    bool fifo_full;
    uint32_t fifo_level;
    uint32_t tile_count;
} lhsi_pc_status_t;
int  lhsi_pc_status(lhsi_pc_handle_t *h, lhsi_pc_status_t *out);
```

Return codes follow the standard Linux convention: `0` on success, `-errno` on failure.

6. Synthesis-Time Configuration

The following parameters are baked in at synthesis time and cannot be changed at runtime. A compiler reads them via the `INFO_*` registers and tailors its code generation accordingly.

A future revision (`pc-isa-0.2`) will expose `THRESHOLD` as a runtime-writable register. The RTL

Parameter	Default	Range (validated)	Notes
BLOCK_M	64	16, 32, 64, 128, 256	Combined product must make N a power of two
BLOCK_N	64	16, 32, 64, 128, 256	(same)
SCORE_WIDTH	8	4, 6, 8, 10, 12, 16	Defines tdata width on s_axis_scores
THRESHOLD	10	**10 only** in this version	The shift-and-add hardware path implements $\times 10$ exactly
FIFO_DEPTH	16	4, 8, 16, 32	Decision output FIFO depth

change is straightforward but breaks the current zero-cost shift-and-add multiplier on the right-hand side of the inequality.

7. Bit-Accurate Reference

The Python model at `sw/reference_model/precision_controller_ref.py` is the canonical bit-accurate reference for this ISA. A compiler can verify its codegen against the model directly:

```
from precision_controller_ref import PrecisionController

pc = PrecisionController()
decision = pc.process_tile(scores)   # exactly what the chip will return
```

The model is verified bit-exact against the RTL testbench's 143 replay tiles (see `sw/reference_model/test_precision_controller_ref.py`). Any divergence between the model and the chip is a bug in this specification, not in the chip.

8. Integration Phases

This specification supports a staged integration that begins before silicon exists. The interface described in this document is the stable contract through every phase.

Phase	Timeline	Compiler targets	We provide
0	now	Bit-accurate Python reference model	Model + ISA + tests on the `compiler-integration-prep` branch
1	when ZCU102 / 104 arrives	AXI-Lite + AXI-Stream on Zynq UltraScale+	Vivado bitstream + PYNQ driver
2	after KV-cache, token-importance, and memory-controller blocks complete	Same AXI interface, more blocks visible	Multi-block FPGA project
3	post-tape-out (2027+)	PCIe-attached custom chip	TSMC 16FFC silicon + Linux driver

9. Open Questions

The following are intentionally on the table for collaborators. None of them are committed in `pc-isa-0.1`.

1. Should `THRESHOLD` be elevated to a runtime register? Cost: one register and a parameterized multiplier in place of the current shift-and-add. Benefit: per-workload tuning without re-synthesis.
2. Is a single-bit decision stream sufficient, or do we want to also emit the raw `max` and `sum` values for compiler-side calibration?
3. Should the decision FIFO support coalesced reads (e.g., 32 decisions packed into one 32-bit word) for high-throughput batching?
4. Are AXI-Lite + AXI-Stream the right interfaces, or should we expose a CHI / TileLink / custom protocol that matches the rest of the chip?
5. Should the block expose a small reference-counting feature so the compiler runtime can track tile-to-decision mapping when multiple attention streams are in flight?

10. Change Log

- `pc-isa-0.1` (2026-05-13): First public draft. Stable for the precision controller block only.

LonghornSilicon Adaptive Precision Attention — ACU Interface Specification. This document is open and may be redistributed. The TSMC 16FFC PDK referenced in the integration phases is NDA-protected and is not part of this document or the public repository.